

LLM - Foundation

tokens: takes prompt $\rightarrow$ chop it into tokens

$\rightarrow$ full words, prefixes, suffixes
character

$\rightarrow$ your input becomes
number

vectors = $[1.632, 382, 103]$



- Embeddings: to make tokens have
meanings, tokens that are related

will have similar values / coordinate

- Self-Attention: $[Q, k, v]$ will make

the model Learn From previous ones

in Long Range

↳ Attention = $\text{softmax}(Qk / \sqrt{d-k})$

↳ strict Attention:

// "No Looking to Future" //

A B C D $\rightarrow$ A B $\leftarrow$ C $\rightarrow$ D

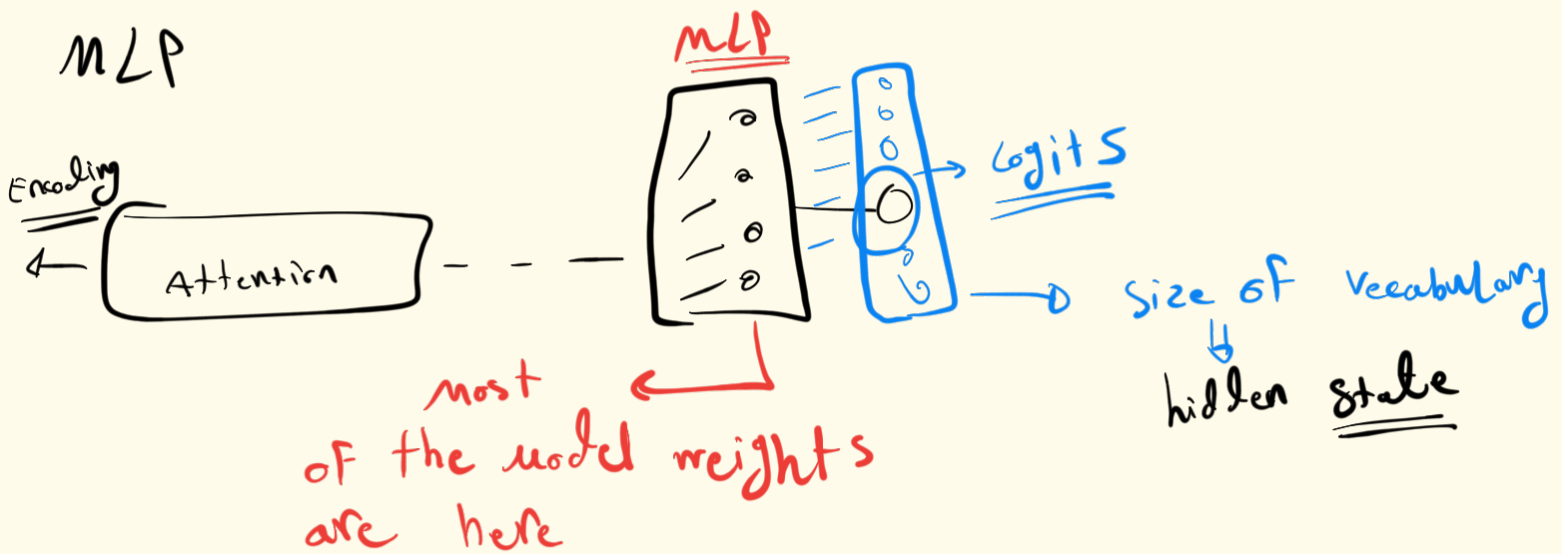
multihed Attention: Different info $\rightarrow$

in Parallel: (Long-Range, Code, List Str.)

↳ Classic MH $\rightarrow$ Problem (extensive during inference)
↳ each one has their own Q, K, V

Solution: MQA, GQA $\rightarrow$ grouped query
- shared key, value
↳ many query
- shared key/value

Fewer KV-Heads $\rightarrow$ Smaller K-v cache $\rightarrow$ better scalability



Positional Encoding:

Attention alone $\rightarrow$ no order

modern LLM uses
RoPE $\rightarrow$ inject position
by Rotation Q, K Vectors

★
* Long context behavior is partly constrained by PE design

Soft max & Turn logits $\rightarrow$ Probability

$\rightarrow$ Decoding & chooses next tokens
common decoding method

Logits $\rightarrow$ $\left[\begin{array}{l} \text{greedy, top-k, top-p} \\ \text{temperature sampling} \end{array} \right]$
 $\downarrow$
we only see tokens
to the left
So the prob of choosing
will be from these
method

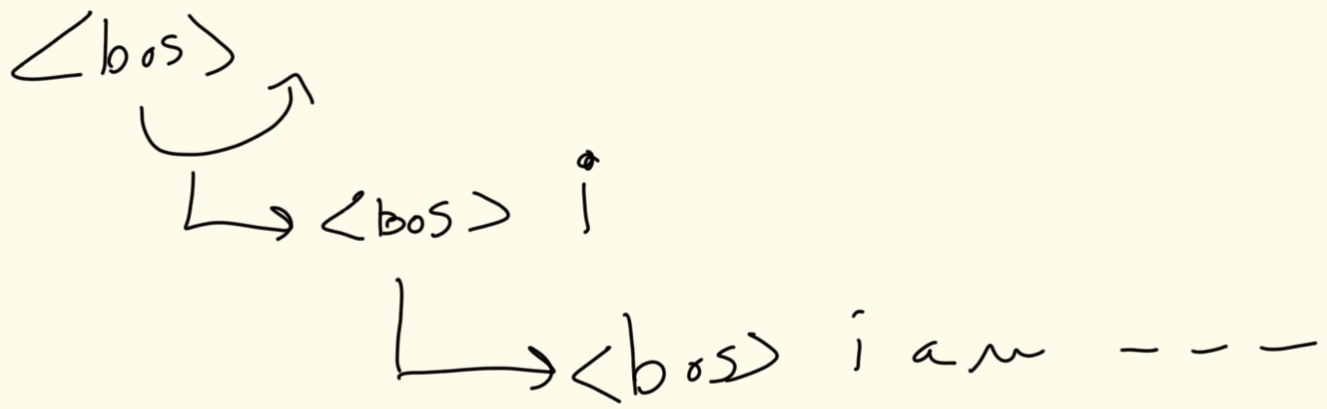
Kv cache $\rightarrow$ store the k, v's
before starting $\rightarrow$ safe time

what is (Kv cache) ??

$$\alpha_{ij} = q_i \cdot k_j$$

in training this efficient because with
GPU i can process them at once since
i have full sequence of words
start $\longrightarrow$ END

* The problem in inference we don't have

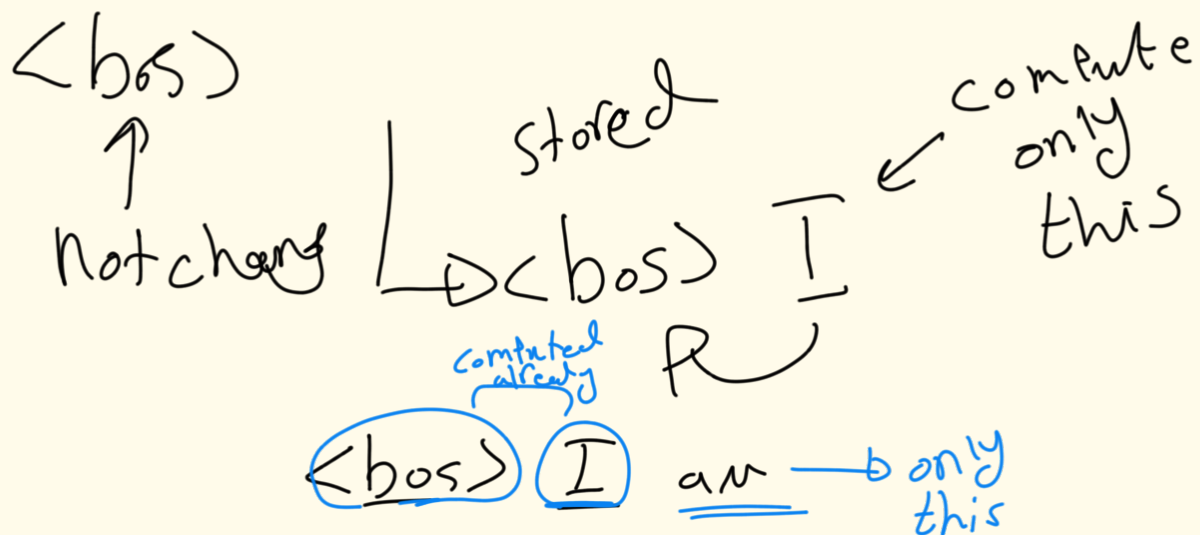


So we compute kv every single
time $\rightarrow$ inefficient in inference

kv cache

$\hookrightarrow$ we store the computed kv

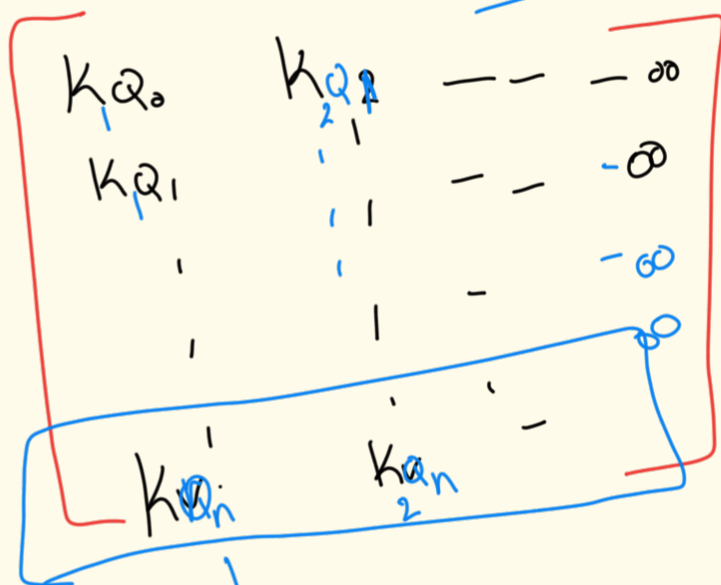
Since



Caching happening only for
 (key, value)

$Q \Rightarrow X$
 we don't need
 previous query

Casual Self Attention ($N \times N$)



without Caching
 It's Quadratic
 Growth

we need only this IF we use
 Caching $\rightarrow$ Linear n Sequence

Growth

* $\rightarrow$ we need a

i don't need every query
 since we cache only position

* Last Query for last tokens k, v
 only

* keys for All tokens

without kv cache

kv cache

Project
Cost

$$O(N \cdot d^2)$$

↳ input ($N \times d$)
↳ weights ($d \times d$)

$$O(d^2)$$

↳ ($1 \times d$)
↳ ($d \times d$)

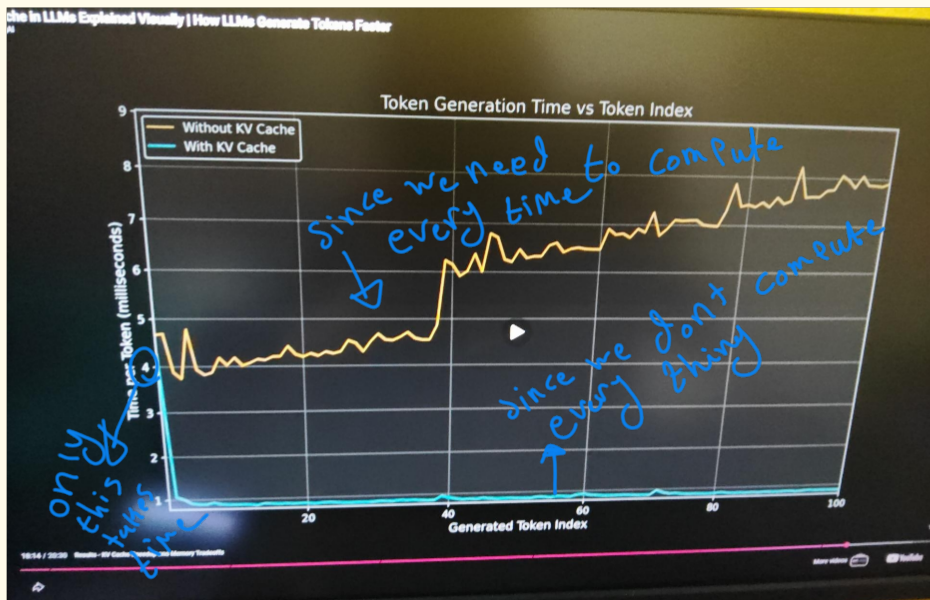
Attention
(QK)

$$O(N^2 \cdot d)$$

$(N \times d) \cdot (d \times N)$

$$O(N \cdot d)$$

$(1 \times d) \cdot (d \cdot N)$





Compute ↓ memory ↑

Trade offs

sequence length n

$$\text{memo} = L \times H \times d_h \times \frac{1}{T} \times 2 \times \text{byte}$$

↳ Linearly increasing

$$80 \times 8 \times 128 \times 8192 \times 2 \times 2$$

$$\xrightarrow{B=1} 2.5 \text{ GB} \quad \xrightarrow{B=4} \underline{\underline{10 \text{ GB}}}$$

Just kv cache } { activation
weights

